Rapport de stage

de première année BTS SIO SLAM

effectué du 15 Juillet au 14 Août 2025

Klaudia Juhasz

Table des matières

Introduction....4
Présentation de l'entreprise OPS Imperium....4
Projet....6
Contexte....6
Besoins....6
Travail à réaliser....6
Analyse de l’étude existante et l’étude d’opportunité....7
Étude existante....7
Étude d’opportunité....7
Planning prévisionnel....7
Mise en place et configuration de solutions logicielles....8
Développement et maintenance d'applications....8
Développement....8
Déploiement d’application via Docker....9
Mini-projet n8n « English»....9
Mini-projet « Google Docs API »....10
Projet LinkStream....12
Backend....12
Frontend....16
Gestion de bases de données....18
Tests et validations....19
Support aux utilisateurs....19
Analyse critique des activités....20
Compétences acquises et développées....20
Comparaison avec les attentes initiales....20
Apport du stage et perspectives....20
Critique du planning....21
Bilan des objectifs atteints....21
Apport du stage en termes de compétences et de connaissances....21
Propositions d'amélioration pour l'entreprise....22
Perspectives d'avenir (professionnel, poursuite d'études)....22
Conclusion....23
Synthèse des principaux points....23
Réponse à la problématique....23
Remerciements....24
Annexes....25
Bibliographie....40

Index des figures

Figure 1: Diagramme de Gantt....32
Figure 2: Déploiement via Docker....33
Figure 3: Mini-projet English....33
Figure 4: Mini-projet Google Docs API....34
Figure 5: Workflow Linkedin Chat....34
Figure 6: Workflow register webhook Unipile....35
Figure 7: Subworkflow linkedin_messages....35
Figure 8: Subworkflow linkedin_people....36
Figure 9: Création webhook par Unipile....36
Figure 10: Authentification par Unipile....37
Figure 11: Redirection vers l’application LinkStream....37
Figure 12: Affichage dynamique....38
Figure 13: Affichage des conversations....38
Figure 14: MCD de la base de données Supabase, réalisé avec Looping....39
Figure 15: MCD (proposition d’évolution), réalisé avec Looping....39

Introduction

J’ai eu l’opportunité de réaliser un stage de 5 semaines au sein de l’entreprise OPS IMPERIUM, sous la supervision de son dirigeant, M. Victor Langlois.

Présentation de l'entreprise OPS Imperium

Identité et activité

OPS Imperium est une société par actions simplifiée unipersonnelle (SASU) créée en mai 2025 par Victor Langlois, expert en intelligence artificielle et DevOps avec plus de 15 ans d'expérience. Basée au 3 rue Paganini à Nice, l'entreprise est spécialisée dans l'intelligence artificielle, l'automatisation des processus métiers et l'infrastructure cloud (DevOps). Elle intervient sur l'ensemble de la chaîne de valeur technologique, allant de l'architecture de solutions IA complexes à l'implémentation d'automatisations opérationnelles.

Positionnement et modèle économique

L'entreprise se distingue par un modèle économique hybride innovant. D'une part, elle fournit une expertise de haut niveau en IA et DevOps auprès de grandes entreprises et institutions (Gouvernement de Monaco, Hôpital de Monaco, grands transporteurs européens), à travers des missions complexes d'architecture, d'implémentation et d'accompagnement stratégique. D'autre part, elle propose une offre de solutions accessibles pour les TPE/PME et les entrepreneurs indépendants : automatisations prêtes à l'emploi, formations pratiques et accompagnement personnalisé à la transformation digitale.

Mission et approche

La mission d'OPS Imperium est de démocratiser l'accès à l'intelligence artificielle et à l'automatisation. Face au constat que 69% des TPE/PME françaises n'utilisent pas encore l'IA générative, l'entreprise s'est fixé pour objectif de créer et proposer des solutions pragmatiques et abordables. L'expertise acquise sur des projets complexes est réinvestie le développement d'outils simplifiés, permettant aux petites structures de bénéficier d’une grande partie de la valeur technologique avec une complexité réduite.

Organisation et développement

Actuellement centrée autour de son fondateur-président, OPS Imperium est en phase de croissance structurée. Elle collabore avec la BGE Côte d'Azur dans le cadre d'un programme d'accélération et développe des partenariats stratégiques avec d'autres experts du secteur. Parmi ses projets figurent : le développement d'une marketplace d'automatisations, la création de contenus de formation spécialisés

et, à terme, la constitution d'un collectif d'experts ("Studio IA") pour répondre à des projets d'envergure nationale.

Contexte du stage

Le stage s'inscrit dans cette phase dynamique de développement, offrant une expérience complète au sein d'une startup technologique en pleine croissance. Il a permis de participer à différents volets d'activité: support aux missions de conseil, contribution au développement des solutions logicielles, aide à la structuration des processus internes, et implication dans les actions de communication et de développement commercial.

Description du service/département d'accueil

J'ai été accueillie directement au sein de l'entreprise OPS Imperium, qui concentre l'ensemble de ses activités autour de la recherche, du développement et de l'innovation technologique. L'entreprise n'est pas structurée en départements distincts : toutes les missions gravitent autour de la conception de solutions en intelligence artificielle, de l'automatisation des processus métiers et du déploiement d'infrastructures cloud. Dans ce contexte, mon stage s'est déroulé au cœur de cette dynamique d'innovation, en étroite collaboration avec le fondateur.

Environnement de travail et technologies utilisées

L'environnement de travail était celui d'une petite structure agile, offrant à la fois une grande proximité avec le tuteur et une autonomie significative dans la réalisation des projets. Le cadre m'a permis de participer activement à toutes les étapes, de la conception au développement, dans une atmosphère collaborative et orientée vers l'expérimentation.
Sur le plan technique, plusieurs outils et technologies modernes ont été utilisés :

- **Automatisation :** *n8n* pour la création et l'orchestration de workflows.
- **Bases de données :** *Supabase* (PostgreSQL) pour le stockage et l'accès structuré aux données.
- **Intégration d'API :** connexion avec *Unipile* pour gérer les interactions avec LinkedIn et autres services.
- **Développement frontend :** *Streamlit* (framework Python) pour concevoir une interface simple et interactive.
- **Déploiement :** *Docker* et *docker-compose* afin d'assurer la portabilité et la reproductibilité des environnements.
- **Langages et librairies :** Python (requests, pandas), ainsi que l'utilisation d'API Google (Text-to-Speech, Google Docs API).
- **Outils de développement :** *VS Code* pour le codage et *GitHub* pour le suivi de version.

Cet environnement m'a permis de travailler sur un projet complet en mêlant automatisation, gestion de bases de données, intégration API, développement web et déploiement.

Projet

Contexte

Dans le cadre de l'automatisation des processus métier, l'entreprise souhaite mettre en place un système basé sur ***n8n***, un outil d'automatisation et d'orchestration de workflows. L'objectif est d'exploiter ***n8n*** pour récupérer et traiter automatiquement des données issues d'un réseau social (LinkedIn) (par exemple : récupération de profils, gestion de messages, suivi de contacts, etc.), puis de centraliser et exploiter ces données dans une base de données interne.
Cependant, ***n8n*** ne fournit pas directement d'interface utilisateur ergonomique pour consulter, filtrer ou déclencher des actions sur ces données. Un frontend dédié est donc nécessaire pour exploiter pleinement ces automatisations.

Besoins

- **Centralisation des données** :
  Stocker les informations collectées par ***n8n*** dans une base de données structurée.
- **Visualisation et interaction** :
  Mettre à disposition une interface graphique (frontend) permettant aux utilisateurs de consulter les données issues de LinkedIn et d'autres sources.
- **Pilotage des workflows** :
  Permettre le déclenchement manuel de certains workflows n8n via des boutons ou actions dans l'interface, grâce à des webhooks ou appels à l'API n8n.

Travail à réaliser

Développement Backend & Intégration
Concevoir et configurer les workflows n8n pour :
- Récupérer des données LinkedIn (profils, messages, etc.)
- Stocker ces données dans une base de données (ex. PostgreSQL, MySQL)

Mettre en place les endpoints (API) permettant :
- L'accès aux données stockées
- Le déclenchement des workflows via API ou webhook

Développement Frontend
- Créer une application web affichant les données issues de la base :
  - Listes, tableaux, graphiques
  - Filtres et recherche
- Intégrer des actions utilisateur :
  - Boutons ou formulaires pour déclencher des workflows spécifiques
  - Confirmation et suivi de l'état des automatisations

Architecture & Communication
- Assurer la communication entre le frontend, la base de données et n8n
- Garantir la sécurité des échanges (authentification API, droits d'accès)

Analyse de l'étude existante et l'étude d'opportunité

Étude existante

L'entreprise dispose actuellement d'une infrastructure d'automatisation basée sur n8n permettant de créer et exécuter des workflows pour automatiser certaines tâches liées à Linkedin.

Ces workflows s'appuient sur :

- ***l'API Unipile*** afin de pouvoir accéder aux données LinkedIn comme messages, profils, informations de contact…
- ***la base de données en ligne, Supabase*** pour stocker les données collectées dans une base relationnelle et offrir une API d'accès
- ***n8n Cloud*** dans le but d'orchestrer les workflows et interagir avec des autres services grâce aux webhooks et API

Cependant ce projet a été lancé mais n'avait jamais été encore finalisé, car premièrement il n'y a aucune trace exploitable dans la base de données existante liée à ce projet, deuxièmement le système ne dispose pas d'interface utilisateur dédiée, ce qui empêche l'utilisation des données collectées et l'automatisation de la base de données.

Étude d'opportunité

La finalisation de ce projet offrira plusieurs avantages à l'utilisateur, comme :

- avoir un accès simplifié pendant la consultation de ses messages et des profils Linkedin depuis une interface claire et intuitive, sans connaissances techniques,
- gagner du temps en évitant les manipulations techniques afin d'afficher les données ou lancer un workflow,
- utiliser un outil centralisé comme la consultation, le filtrage et envoi de messages LinkedIn regroupés dans une seule application ;
- contrôler manuellement le déclenchement des automatisations n8n directement depuis cette interface.

En résumé, sa mise en place est une opportunité quand elle permet d'améliorer rapidement l'expérience utilisateur et de préparer des évolutions futures comme l'ajout de notifications ou l'intégration avec d'autres réseaux sociaux.

Planning prévisionnel

Un planning prévisionnel est un outil de planification qui permet d'organiser les différentes étapes d'un projet avant son exécution. Il définit la répartition des tâches dans le temps, identifie les dépendances entre elles et donne une vision d'ensemble de l'avancement attendu.
Sous forme de diagramme de Gantt, il facilite le suivi du projet en indiquant la durée prévue de chaque activité, les jalons importants et les périodes de chevauchement éventuel.

Dans le cadre de mon stage, un planning prévisionnel a été établi afin de structurer les différentes phases de travail : formation initiale, réalisation de mini-projets, développement backend et frontend, puis tests et validation. Ce document a servi de fil conducteur tout au long du projet.

Le diagramme de Gantt correspondant se trouve en *Figure 1 : Diagramme de Gantt* de ce rapport.

Mise en place et configuration de solutions logicielles

Dans le cadre de ce projet, j'ai réalisé la mise en place et la configuration des différents éléments logiciels qui sont nécessaires au fonctionnement de cette application :

- **n8n :**
  création, paramétrage des workflows, configuration des webhooks et intégration avec les API externes
- **Supabase :**
  création des fonctions stockées et des requêtes, configuration de l'API
- **API Unipile :**
  configuration des accès pour la récupération et l'envoi de données LinkedIn
- **Application Streamlit :**
  installation des extensions nécessaires, configuration des clés API, comme n8n, Supabase, Unipile

Cette étape a permis de cordonner tous les composants (frontend, base de données et workflows) et d'obtenir un système (presque) fonctionnel, (quasiment) prêt à être utilisé et amélioré.

Développement et maintenance d'applications

Développement

Avant de commencer le développement de cette application, la première semaine a été consacrée à me former sur l'utilisation de Docker, l'automatisation et l'intégration API dans du code Python.
Durant cette période j'ai:

- effectué des recherches sur la notion d'*« automatisation »*, sur le concept **d'*Agentic Factory***, ses fonctionnalités et ses cas d'utilisation.
- parcouru le site de ***n8n*** afin de consulter la documentation et découvrir l'utilisation de chaque type de nœud.
- recherché de la documentation sur ***Google Cloud*** afin de comprendre comment utiliser des APIs dans un programme implémenté en Python.
- visionné plusieurs tutoriels sur YouTube.

Déploiement d'application via Docker

Figure 2 : Déploiement via Docker

Pour faciliter le déploiement et assurer un environnement de développement homogène, j'ai utilisé ***Docker***. L'objectif était de contenir l'application dans un serveur web basé sur *Nginx*, afin qu'elle soit exécutable de manière isolée, reproductible et portable entre systèmes d'exploitation.

Il y a deux manières de créer une *image Docker* :

Utilisation de docker-compose :
On peut créer un fichier *docker-compose.yml* à la racine de notre application, qui décrit la configuration du service web :

- il construit une image nommée *« mini-crm »* à partir d'un Dockerfile.
- il expose le port 80 du conteneur sur le port 8080 de l'hôte, ce qui permet d'accéder à l'application.
- il monte les dossiers locaux (*html, js et image)* dans le conteneur grâce aux volumes, ce qui permet de modifier les fichiers sans reconstruire l'image.
- il définit une politique de redémarrage *« unless-stopped »,* afin d'assurer que le service redémarre automatiquement sauf arrêt manuel par l'utilisateur.

Pour lancer l'application, il suffit d'exécuter la commande *« docker-compose up –build »* dans le terminal. Cela construit l'image et démarre le conteneur, ce qui rend l'application accessible en local.

Création d'une image distribuable avec Dockerfile
Si l'on souhaite fournir une image *« fixe »* et autonome telle que déployer sur une autres machine sans partager les volumes, il est nécessaire de définir un *Dockerfile*, lequel indique à ***Docker*** comment construire l'image de l'application. *(cf Annexe 1 : Déploiement via Docker)*

De cette manière, l'application est empaquetée directement dans l'image et peut être déployée sur n'importe quelle machine Docker sans dépendre des fichiers locaux.

Mini-projet n8n « English»

Figure 3 : Mini-projet English

L'objectif de ce mini-projet était de créer un *POC (Proof of concept)* d'une automatisation avec n8n.
J'ai choisi de développer un workflow permettant de recevoir, chaque soir, une courte histoire en anglais (max. 150 mots) générée par IA, accompagnée d'un fichier audio au format MP3 afin de permettre de s'entraîner à la compréhension orale.

N8N est un outil d'automatisation open source qui permet de créer des workflows visuels pour relier des applications, des bases de données, des API et des services externes.
Il se base sur une logique de ***nœuds (nodes)***, qui représentent chacun une action ou un traitement particulier (ex. : déclencheur, transformation de données, appel d'API, envoi d'e-mail).

Pour ce projet, j'ai utilisé des nœuds que vous pouvez consulter dans *l'Annexe 2.*

Tests et validations

Afin d'assurer la fiabilité du workflow, une procédure de test a été mise en place :

- ***Tests unitaires des nœuds :***
  - Input : données minimales nécessaires (texte de test pour Text-to-Speech).
  - Output attendu : résultat conforme (MP3 encodé généré correctement).
  - Validation : inspection de la sortie JSON dans l'interface n8n, vérification des champs attendus.
- ***Test de bout en bout :***
  - Déclenchement manuel du workflow complet.
  - Vérification que l'email reçu contient bien :
    - le texte généré (maximum 150 mots),
    - le fichier MP3 lisible.
  - Contrôle que l'histoire est en anglais et cohérente avec la consigne initiale.
- ***Critères de validité :***
  - Respect de la limite de mots.
  - Fichier audio lisible sans corruption.
  - Déclenchement à l'heure prévue par le Schedule Trigger.
  - Aucune erreur de connexion (API, clé d'authentification).

Analyse critique des activités

J'avais initialement estimé un temps de réalisation de 3 à 4 heures. Ce délai n'a pas été respecté pour deux raisons principales :

1. **Choix du modèle d'IA :**
   - La plupart des modèles de type "chat" sont payants.
   - Après plusieurs recherches, j'ai trouvé OpenRouter, qui permet d'accéder gratuitement à certains modèles IA.
2. **Ajout de la fonctionnalité audio :**
   - L'envoi du texte seul était relativement simple à mettre en place.
   - L'ajout d'une conversion *text-to-speech* a nécessité de trouver une API gratuite, puis de configurer correctement l'authentification.
   - La solution finale a consisté à placer la clé d'authentification directement dans l'URL au lieu de l'envoyer dans les *headers*.

Ces difficultés ont doublé le temps total de réalisation, portant la durée effective du projet à environ 8 heures.

Mini-projet « Google Docs API »

Figure 4 : Mini-projet Google Docs API

L'objectif de ce mini-projet est de créer automatiquement, en mode *« serverless »*, un document Google Docs contenant des champs dynamiques (par exemple : *{{NOM}}, {{EMAIL}}*), servant de modèle à remplir, puis de lire le contenu de ce document pour analyser les données saisies.

Pour cela j'ai développé les fonctions suivantes en Python :

- **token()** : authentifie l'utilisateur via *token.json* ou *credentials.json.*
- **creation_google_doc()** : crée un nouveau document Google Docs intitulé *« Formulaire Test »* et y insère du texte avec des champs dynamiques (*Nom : {{NOM}}*).
- **extraits_champs()** : lit le document et récupère soit les champs dynamiques (lors de la création), soit le texte complet (pour analyser après modification).
- **champs_modifiés()** : identifie les champs non remplis et extrait les valeurs saisies par l'utilisateur.

Tests et validations

Chaque fonction a été testée individuellement pour vérifier qu'elle remplissait correctement la tâche attendue.

Procédure de test mise en place :

- ***Tests unitaires des fonctions :***
  - Input : appel d'une fonction avec des données de test (par exemple : document contenant *« Nom : {{NOM}} »*
  - Output attendu : comportement conforme (par exemple : récupération correcte des champs dynamiques).
  - Validation : inspection du retour de fonction et comparaison avec le résultat attendu.
- ***Test de bout en bout :***
  - Création d'un document ***Google Doc***s via *« creation_google_doc() »*.
  - Modification manuelle du document (saisie de Nom, Email, etc).
  - Lecture du document avec *« champs_modifiés() »* pour vérifier que les données saisies sont correctement extraites.
  - Contrôle que les champs non remplis apparaissent bien comme manquants.
- ***Critères de validation :***
  - Authentification réussie sans erreur.
  - Création du document avec insertion des champs dynamiques.
  - Extraction correcte des champs avec prise en compte des valeurs remplies et des champs vides.
  - Résultat conforme aux attentes même en cas de variation dans les intitulés (par exemple : Prénom vs PRENOM).

Analyse critique des activités

J'avais initialement estimé un temps de réalisation de 3 heures. En réalité, la durée a été de 6 heures en raison de deux principales difficultés :

1. Authentification :

- Identifier la bonne méthode pour authentifier l'utilisateur.
- Utilisation de *credentials.json* pour générer un premier token, puis enregistrement de ce token pour les authentifications futures.

2. Récupération des données dans *champs_modifiés()* :

- Les résultats initiaux montraient :

NOM : {{JUHASZ}}

PRENOM : (introuvable)
EMAIL : {{juklau@hotmail.fr}}
ENTREPRISE : (introuvable)

- Le problème provenait d'un décalage entre les noms de champs attendus (ex. *PRENOM, ENTREPRISE*) et ceux écrits dans le document (*Prénom avec accent, Société*)
- Solution : créer un dictionnaire de correspondance et adapter l'expression régulière (*regex*).

```
# par exemple, intitule = 'Prénom' si champ = 'PRENOM'
intitule = correspondance[champ]
pattern = rf"{intitule}:\s*(.+)"
```

Perspectives
Ce travail pourrait servir de base pour automatiser la création de formulaires internes, de contrats ou pour extraire et suivre les champs remplis par différents utilisateurs.

Projet LinkStream

Backend

La première étape a consisté à découvrir l'écosystème afin de comprendre les outils et services impliqués.

J'ai consulté la documentation de *l'API Unipile* (relative aux chats, messages et profils) pour comprendre comment cette API structure les conversations LinkedIn, ainsi que les champs disponibles et obligatoires associés à un chat ou à un message.

Unipile utilise une API unifiée pour gérer les conversations LinkedIn de manière structurée et automatisée. Toutes les conversations (LinkedIn classique, Sales Navigator, Recruiter) sont centralisées dans une seule boîte de réception.
Chaque chat est identifié par un « *chat_id » ;* les participants sont listés via leurs *« attendees_ids ».*
Les messages sont rattachés à un chat et possèdent un *« message_id »*, un *« timestamp »* et un identifiant d'expéditeur (*« sender »*).

Les tables *Supabase* fournies (chats, messages, people) ont ensuite été examinées afin d'en analyser la structure, les champs et leurs relations.

Supabase est une plateforme open source permettant de créer facilement des backends pour des applications web ou mobiles. Basée sur PostgreSQL, elle intègre notamment :

- **Base de données :** gestion des tables, colonnes et données.
- **Authentification :** gestion des utilisateurs, connexions sociales (Google, GitHub, etc.), rôles et permissions.
- **Stockage de fichiers :** hébergement des images, vidéos, documents, etc.
- **API en temps réel :** synchronisation instantanée des données entre les utilisateurs et l'application.
- **Fonctions serverless :** exécution de code backend sans gestion de serveur .
- **Vector embeddings :** intégration de modèles d'IA pour la recherche intelligente.

J'ai ensuite identifié ***le flux de données*** afin de comprendre le fonctionnement backend de l'application :

1. **Unipile → n8n :** Capture et déclenchement
   - ***Unipile*** agit comme une passerelle API unifiée pour des plateformes comme LinkedIn, WhatsApp, Gmail, etc.
   - Lorsqu'un événement survient (nouveau message LinkedIn, invitation acceptée), un webhook est envoyé vers ***n8n***.
   - Dans n8n, un ***nœud Webhook*** reçoit ces données en temps réel.
   - Les données sont généralement transmises ***au format JSON***, prêtes pour traitement.
2. **Traitement dans n8n :** Automatisation et enrichissement
   - Utilisation de nœuds de transformation pour filtrer, reformater ou enrichir les données.
3. **n8n → Supabase :** Stockage structuré
   - Envoi des données traitées vers ***Supabase*** via le nœud dédié.
   - Création, mise à jour ou suppression d'entrées en base.
   - Interrogation préalable pour éviter les doublons ou compléter les informations.
   - Supabase assure le stockage dans PostgreSQL tout en offrant des fonctionnalités avancées (authentification, stockage, vectorisation IA, etc.).

Création du worflow n8n

Initialement, il était prévu que le workflow soit fourni afin de peupler les tables de la base de données. Cependant, l'importation du ***fichier JSON*** correspondant dans n8n a échoué. J'ai donc ***repris la création du workflow*** depuis le début, en m'appuyant partiellement sur les éléments présents dans ce document JSON.

Pour visualiser le flux de données et structurer le processus, j'ai d'abord créé un workflow principal, puis j'ai découpé l'ensemble en trois grandes étapes à l'aide de *subworkflows :*

1. ***Le workflow parent :*** récupération et traitement de chaque message.
2. ***Un subworkflow*** pour récupérer les ***messages*** de chaque chat (exercice d'automatisation).
3. ***Un subworkflow*** pour traiter les ***personnes concernées*** (récupération des noms, photos LinkedIn, etc.).

Ci-dessous se trouve la présentation du résultat final. Durant la conception, j'ai testé plusieurs types de nœuds et différentes approches, afin d'obtenir un workflow fonctionnel capable de récupérer, traiter et stocker les éléments de chaque chat dans la base de données.

1. Construction du workflow de Chats

Figure 5 : Workflow Linkedin Chat

L'objectif est de créer un workflow n8n permettant de récupérer et de stocker les conversations.

Pour ce workflow, j'ai utilisé des nœuds présentés dans *l'Annexe 3.*

Difficultés rencontrées et solutions apportées :

a) Transtypage du champ *« folder »*
Problème : le champ était de type *string*, ce qui le rendait incompatible avec les étapes suivantes du workflow.
Solution : conversion en tableau via un nœud fonction :*« {{[$json.folder[0]]}} »*.

b) Traitement limité à un seul chat
Problème : seul le premier chat était traité.
Solution : ajout d'une boucle permettant de traiter chaque chat individuellement.

c) Vérification de la présence d'un chat dans la base
Problème : nécessité d'éviter les doublons lors de l'insertion.
Solution : requête SQL pour récupérer le *« row_id »* (identifiant unique généré automatiquement) et de déterminer si le chat était déjà présent.

d) Récupération du *« row_id »* après insertion
Problème : nécessité de transmettre le *« row_id »* du chat nouvellement inséré pour lier les messages correspondants. Sans cette valeur, le workflow ne pouvait pas fonctionner correctement.
(lié avec le problème « Affichage des conversations » page 19).
Solution : récupération du *« row_id »* via une requête SQL, puis utilisation d'un nœud *Merge* pour transmettre la valeur au reste du workflow.

e) Exécution du workflow
Problème : après le boucle d'insertion des chats, le workflow s'arrêtait et il fallait le redémarrer pour insérer les messages dans la base.
Solution : création et appel d'un subworkflow dédié au traitement et à l'insertion des messages.

f) Automatisation de l'enregistrement des conversations :
Problème : nécessité d'un enregistrement automatique des conversations de l'utilisateur dans *la table linkedin_chats.*
Solution : création d'un webhook dans le workflow des chats exploitant ***l'API Unipile.*** Après authentification de l'utilisateur, ses conversations sont enregistrées automatiquement. L'automatisation est déclenchée par l'appel de ce webhook depuis l'application.
(détails précisés dans la section « Connexion à la base Supabase et récupération des messages » page 18).

g) Utilisation d'un webhook :
Problème : en configurant les paramètres d'authentification directement dans la section *Send Headers* du nœud, l'authentification n'a pas fonctionné.
Solution : utilisation du mécanisme *Generic Credential Type*, permettant de créer un *Header Auth*. Cette configuration a permis d'assurer l'authentification correcte du webhook.
Figure 6 : Workflow register webhook Unipile

2. Construction du subworkflow de messages- Extension aux Messages

Figure 7 : Subworkflow linkedin_messages

L'objectif est de compléter le workflow précédent afin de récupérer et stocker les messages de chaque conversation.

La création de ce subworkflow m'a permis de constater son utilité, en particulier pour faciliter le traitement des données et gérer la récupération en cascade (chat → messages). Sa logique est similaire à celle du workflow parent.

Pour ce workflow, j'ai utilisé des nœuds présentés dans *l'Annexe 4*.

Difficultées rencontrés et solutions apportées :

a) Récupération du *row_id* du chat
Problème : sans ce champ, il était impossible de lier les messages à leur chat.
Solution : récupération du *row_id* directement après l'insertion du chat dans Supabase, puis transmission via un nœud *Merge* dans l*e workflow des chats.*

b) Insertion des messages
Problème : lors de l'utilisation d'un nœud *HTTP Request*, un seul message était inséré alors que le journal d'exécution indiquait 21 insertions.
Solution : remplacement du nœud *HTTP Request* par le nœud *Supabase*, qui a permis d'assurer une insertion correcte des messages.

c) Gestion des doublons de messages
Problème : initialement, j'avais créé un identifiant local (« *local_id »)* pour les messages, en complément de l'id généré par LinkedIn (« *linkedin_message_id »*). Toutefois, comme les deux ne correspondaient pas, cela a entraîné des doublons dans la base. Cette erreur m'a coûté environ une journée de recherche.
Solution : abandon de ce mécanisme. Désormais, les messages sont envoyés via LinkedIn, puis récupérés grâce à l'API Unipile par le webhook du workflow des chats. En cas de nouveau message entrant ou sortant, une mise à jour (*update*) est effectuée directement, ce qui évite les doublons.

d) Automatisation de l'enregistrement des messages dans la base de données :
Problème : nécessité de mettre en place un enregistrement automatique des messages entrants et sortants dans la table *linkedin_messages* afin de gérer en temps réel des messages.
Solution : création d'un webhook dans le *workflow des chats*. Côté *Unipile*, une configuration de suivi des messages a été mise en place : à chaque détection d'un nouveau message, celui-ci est envoyé automatiquement vers le webhook, permettant ainsi son enregistrement en base.

3. Construction du subworkflow de people

Figure 8 : Subworkflow linkedin_people

L'objectif de ce subworkflow est de récupérer et d'enrichir les profils dans la base de données afin de pouvoir les exploiter lors de l'affichage dans l'application (par exemple : photos, noms des participants, accès à la page LinkedIn de l'utilisateur).
La logique suivie est similaire à celle du *workflow parent.*

Pour ce workflow, j'ai utilisé des nœuds présentés dans *l'Annexe 5*.

Difficultés rencontrés et solutions apportées :

a) Insertion des données d'une personne
Problème : les données d'une personne dont le nom contenait un caractère accentué (par ex. « é » dans c*édric*) n'étaient pas insérées correctement dans la base.
Solution 1 (temporaire) : remplacement (en utilisant un nœud) par la "normalisation" cédric en "cedric". Cette solution a permis l'insertion mais a généré des erreurs résiduelles.
Solution 2 (définitive): remplacement du nœud *HTTP Request* par un nœud *Supabase* pour l'insertion. Cette solution a résolu le problème et permis d'insérer correctement tous les utilisateurs, y compris ceux avec des caractères accentués.

Frontend

Avant de commencer à travailler sur l'affichage, j'ai réalisé un état de l'art rapide des technologies de frontend : langages les plus utilisés, frameworks adaptés à chaque langage…
(cf Annexe 6 : Document comparatif des technologies et frameworks)

Pour ce projet, l'objectif étant de créer une application web simple permettant de visualiser les données d'une base Supabase et de déclencher des automatisations via n8n, j'ai retenu les technologies suivantes :

- **Python :** un langage accessible et puissant, adapté à la manipulation de données et à la communication avec des services comme Supabase via API.
- **Streamlit :** framework léger en Python, permettant de développer rapidement des interfaces web interactives sans recourir à HTML ou JavaScript.
- **VS Code :** éditeur de code moderne, bien intégré avec Python et Streamlit, facilitant le développement local et la gestion du projet sur GitHub.
- **Pandas :** librairie incontournable pour la manipulation de données (DataFrames, Series), très performante et disposant d'une documentation riche.

Ce choix technologique permet de construire une application rapidement, de la faire évoluer facilement, et de rester agile si le projet prend de l'ampleur.

Développement d'une application Streamlit pour l'affichage et la gestion des messages LinkedIn

Les premiers pas :

- Installation de l'environnement de développement (***Streamlit***, ***Supabase***, ***pandas***).
- Création d'un environnement virtuel *(« python3 -m venv venv »)* et activation systématique (« *venv\Scripts\Activate\ps1* ») afin d'éviter les conflits de dépendances, garantir la stabilité et assurer la reproductibilité.

Les fonctionnalités développées

a) Authentification via Unipile

LinkedIn ne permettant pas une authentification simple, l'authentification a été réalisée via *l'API Unipile* qui joue le rôle intermédiaire fiable et sécurisé, garantissant que l'utilisateur est bien propriétaire du compte :

- Implémentation d'une fonction *« generate_auth_link() »* qui génère un lien d'authentification temporaire (15 minutes) via l'API Unipile *Figure 9 : Création webhook par Unipile*.
- Une fois que l'utilisateur clique sur ce lien et termine l'authentification, son *account_id* est récupéré grâce à la fonction « *get_account_id_from_unipile() »*.
- Transmission de cet « *account_id* » à n8n via « *post_auth_data_n8n() »* pour automatiser le suivi.

b) Connexion à la base Supabase et récupération des messages

Pour cette étape, les actions ci-dessous décrivent le flux de connexion et d'extraction des données:

- Mise en place d'une fonction *« init_connection() »* pour établir une connexion sécurisée avec Supabase (URL + clé API).
- Récupération des messages via une fonction SQL stockée « *(rpc("get_messages")) »*.
- Transformation des résultats en DataFrame Pandas pour faciliter le filtrage et l'affichage.

c) Gestion du profil utilisateur

Pour personnaliser l'application et permettre à l'utilisateur de visualiser son identité et son réseau :

- Extraction des informations, comme nom, photo de profil, lien LinkedIn depuis les messages envoyés par l'utilisateur.
- Affichage de ces données dans l'*interface Streamlit* avec un design simple et clair.

d) Affichage des conversations

Reproduction d'une interface familière, proche des messageries classiques, et rend la lecture intuitive :

- Identification des conversations par « *chat_row_id »*.
- Création d'une correspondance *nom du contact → id de chat* pour la sélection via un « *selectbox »*.
- Affichage chronologique des messages (grâce au tri sur « *created_at »*) avec un rendu type messagerie ("bulle de chat" : photo, nom, horodatage, contenu du message).

e) Envoi de messages LinkedIn

L'objectif est de transformer l'application en outil interactif et non seulement en tableau de lecture. L'utilisateur peut donc gérer ses conversations de bout en bout :

- Développement de « *post_message_to_linkedin() »*, qui envoie une requête POST à l'API Unipile pour transmettre un message dans une conversation existante.
- Intégration d'un champ de saisie (« *st.chat_input() »)* pour l'écriture de nouveaux messages.
- Rafraîchissement automatique de l'interface après envoi pour afficher le message ajouté.

Difficultés rencontrées et solutions apportées:

a) Authentification LinkedIn : *Figure 10 : Authentification par Unipile*
Problème : le flux OAuth complexe.
Solution : utilisation de la logique spécifique d'Unipile via *l'assistant d'authentification hébergé*.

b) Redirection après l'authentification : *Figure 11 : Redirection vers l'application LinkStream*
Problème : impossible de rediriger l'utilisateur vers *« success_redirect_url ».*
Solution : gestion via deux boutons :
- *« Cliquez ici pour vous authentifier »* → redirection vers *« l'assistant d'authentification hébergé »* puis vers la page d'accueil
- *« J'ai terminé l'authentification »* → affichage des conversations.

c) Synchronisation des données :
Problème : requêtes répétées inutilement.
Solution : utilisation « *@st.cache_resource »* et « *@st.cache_data »* pour optimiser les performances.

d) Affichage dynamique : *Figure 12 : Affichage dynamique*
Problème : Streamlit ne gère pas certains composants (badges, styles personnalisés).
Solution : utilisation du HTML et CSS embarqué pour améliorer l'UI.

e) Gestion des données incomplètes :
Problème : certaines colonnes manquaient (photo, nom, etc.).
Solution : création des fonctions *sécurisées comme « safe_first »* pour éviter les erreurs.

f) Identification du bon interlocuteur :
Problème : ma photo LinkedIn apparaissait partout à la place du contact.
Solution : jointure entre linkedin_chats et linkedin_people dans Supabase (*left join entre attendee_provider_id de linkedin_chats et id de linkedin_people*).

g) L'exécution continue de l'application
Problème : l'interface se rafraîchit en boucle.
Solution : déplacement de *« st.run() »* au bon endroit dans *« if prompt ».*

h) Affichage des conversation : *Figure 13 : Affichage des conversations*
Problème : aucune conversation affichée, car *chat_row_id »* n'était pas transmis.
Solution : récupération du *« row_id »* du chat dans le subworkflow linkedin_messages (*« {{$"WhenExecuted by Another workflow').item.json.row_id}} »*).

Gestion de bases de données

Dans le cadre du projet, la gestion de la base de données a constitué une étape essentielle, permettant d'assurer un stockage structuré, cohérent et facilement exploitable par l'application Streamlit.

- ***Recensement des données :***
  Identification des informations à stocker, telles que les messages LinkedIn, les profils d'utilisateur, les conversations, métadonnées comme *dates, chat_id, auteur, expéditeur…*

- ***MCD (Modèle Conceptuel de Données) :***
  Conception d'un schéma mettant en relation les entités principales (Utilisateurs, Conversations, Messages), leurs attributs (*nom, photo_url, contenu du message, sender…*) et leurs relations (un utilisateur peut participer à plusieurs conversations, ou une conversation contient plusieurs messages…) *Figure 14 : MCD de la base de données Supabase*
- ***MLD (Modèle Logique de Données) :***
  Traduction du MCD en tables relationnelles compatibles avec PostgreSQL via Supabase :
  - Table *« linkedin_people »* : informations des utilisateurs.
  - Table *« linkedin_chats »* : conversations avec les identifiants des participants.
  - Table *« linkedin_messages »* : messages avec leur contenu, expéditeur, date et autres métadonnées.
- ***Mise en place de la base de données :***
  Création et configuration des tables, gestion des clés primaires/étrangères (réalisée avec l'aide de mon tuteur) et ajout d'une fonction stockée *get_messages()* pour simplifier l'accès aux données.

Tests et validations

Pour garantir la fiabilité du système, plusieurs phases de tests ont été menées :

- ***Tests unitaires :***
  Chaque fonction (connexion à Supabase, génération du lien d'authentification, récupération/envoi de messages LinkedIn) a été testée séparément.
- ***Tests d'intégration :***
  Validation de la communication entre les différents services (Unipile ↔ n8n ↔ Supabase ↔ Streamlit). Par exemple, l'envoi d'un message devait entraîner son stockage automatique en base via le webhook n8n.
- ***Tests utilisateurs :***
  Simulation d'un parcours complet (authentification → sélection d'une conversation → affichage des messages → envoi d'un message).
- ***Validation fonctionnelle :***
  Contrôle que les résultats correspondaient bien aux besoins exprimés : historique complet des messages, affichage des contacts avec photo et nom, envoi de messages en temps réel.

Ces tests ont révélé des problèmes tels que les doublons de messages, les erreurs d'authentification et l'absence de redirection. Ces problèmes ont été corrigés, permettant d'obtenir un prototype fonctionnel.

Support aux utilisateurs

Même si l'application est encore un prototype, une attention particulière a été portée au support utilisateur, car elle valorise le côté "utilisateur final" du projet, pas seulement la technique.

- ***Interface claire et intuitive :***
  Streamlit a été choisi pour permettre aux utilisateurs non techniques de consulter et gérer leurs messages sans difficulté.

- ***Messages d'erreur explicites :***
  En cas de problème (API indisponible, authentification échouée, absence de données), des messages clairs apparaissent à l'écran pour guider l'utilisateur.
- ***Accompagnement direct :***
  Pendant le stage, j'ai également apporté une assistance à mon tuteur pour tester l'outil et recueillir son retour afin d'améliorer l'expérience.

Analyse critique des activités

Compétences acquises et développées

Ce stage m'a permis de consolider mes compétences techniques en :

- automatisation via n8n,
- gestion de bases de données PostgreSQL via Supabase,
- développement d'interfaces web avec Streamlit, Python,
- intégration d'API (Unipile, Supabase, et Google Cloud).

J'ai également renforcé ma capacité à résoudre des problèmes techniques concrets comme authentification, doublons, gestion des erreurs, et à structurer un projet logiciel en workflow clair via des subworkflows.
Enfin, ce stage m'a permis de progresser en autonomie, en organisation et en communication technique.

Comparaison avec les attentes initiales

Au départ, je pensais surtout participer à des tâches de configuration, d'assistance ou à des tâches simples comme travailler sur l'interface de l'application.
Finalement, j'ai réalisé un projet complet : backend (mise en place des workflows, gestion des données, appels API) et frontend (développement d'une interface fonctionnelle), ainsi que les tests utilisateurs indispensables pour un projet opérationnel.
Mes responsabilités ont été plus larges que prévu, ce qui a rendu le stage encore plus enrichissant.

Apport du stage et perspectives

Ce stage m'a permis de comprendre concrètement le cycle de vie d'une application (backend, base de données, frontend, tests) et de travailler sur des problématiques actuelles liées à l'automatisation et à l'exploitation des données.

Mes perspectives professionnelles et personnelles :

- Approfondir mes connaissances en sécurité (authentification, gestion des accès).
- Améliorer mes compétences en optimisation de bases de données (performances, indexation).
- Explorer l'intégration de l'intelligence artificielle dans les applications.

- M'initier davantage à l'industrialisation des workflows (tests automatisés, CI (Intégration Continue)/CD (Déploiement Continue)).

Critique du planning

Le planning prévisionnel, initial a servi de repère, mais il n'a pas toujours été respecté. Certains mini-projets, ou fonctionnalités ont nécessité plus de temps que prévu (le projet *« English »*, *l'API Google Docs*, la récupération des *« row_id »* des chats, ou encore le peuplement de la base de données), en raison de difficultés techniques imprévues.
À l'inverse, certaines tâches se sont révélées plus rapides que prévu (création du workflow *« linkedin_people »*, l'authentification via Unipile, l'affichage avant la création de la fonctionnalité d'authentification...) ce qui m'a permis de compenser partiellement les retards et de réinvestir ce temps dans d'autres développements.

Cette flexibilité m'a permis d'aller plus loin sur certaines fonctionnalités et de mieux comprendre les outils utilisés. Elle m'a également appris à hiérarchiser les priorités et à gérer les imprévus, une compétence essentielle dans la conduite de projets informatiques.
Enfin, une meilleure planification initiale avec des jalons plus précis aurait pu rendre le suivi plus efficace et limiter certains décalages, mais sans expérience préalable, il était difficile d'anticiper tous les imprévus.

Bilan des objectifs atteints

Les objectifs principaux ont été atteints :

- création d'une base de données fonctionnelle,
- mise en place de workflows n8n,
- développement d'un prototype Streamlit utilisable (authentification, affichage et envoi de messages LinkedIn)

Certaines améliorations restent possibles (authentification OAuth plus avancée, UI plus personnalisée, plus performante), mais la base du projet est solide.

Apport du stage en termes de compétences et de connaissances

En termes techniques, j'ai acquis une réelle expérience dans l'*intégration d'API, le déploiement via Docker, la modélisation et la manipulation de bases de données, l'automatisation avec n8n* ainsi que le *prototypage* rapide d'*applications web avec Streamlit et Python*.

Sur le plan professionnel, ce stage m'a permis de développer des compétences en :

- ***gestion de projet :***
  planification des tâches de développement (ex. création et test des workflows n8n), suivi de l'avancement et adaptation du planning en cas d'imprévus techniques.
- ***communication technique :***
  présentation régulière de mes avancées à mon tuteur, avec des explications claires sur les problèmes rencontrés (ex. doublons de messages, erreurs d'authentification) et les solutions mises en place.

- ***adaptation à un environnement en constante évolution :***
  prise en main rapide de nouveaux outils (n8n, Supabase, Unipile, Streamlit), souvent sans expérience préalable, et recherche de solutions efficaces face aux contraintes et changements de priorités.

Propositions d'amélioration pour l'entreprise

- ***Amélioration de la documentation et des guides internes :*** créer des fiches ou tutoriels pour expliquer les workflows n8n, l'usage de la base de données, et les processus de tests.
- ***Suivi de projet structuré :*** définir un planning avec jalons intermédiaires pour anticiper les retards et mieux répartir les tâches.
- ***Extension à d'autres plateformes*** via Unipile (Gmail, WhatsApp, Instagram, Microsoft, Messenger) pour en faire un véritable hub de messagerie.
- ***Évolution fonctionnelle de l'application :***
  - distinguer les organisations des utilisateurs physiques pour afficher correctement leurs informations,
  - gérer l'affichage des profils sans photo,
  - améliorer le rafraîchissement automatique des messages,
  - intégration d'intelligence artificielle générative (suggestions de réponses),
  - ajouter la recherche et le filtrage avancés dans les conversations,
  - améliorer l'ergonomie et le design (interface réactive, thèmes personnalisés),
  - mettre en œuvre un MCD plus performant *Figure 15 : MCD (proposition d'évolution)*

Perspectives d'avenir (professionnel, poursuite d'études)

Ce stage m'a permis de découvrir concrètement le potentiel de l'automatisation et les possibilités offertes par l'intelligence artificielle, un domaine qui m'intéresse particulièrement.

À l'avenir, je souhaite continuer à me spécialiser en *développement backend*, notamment en approfondissant mes *connaissances sur n8n* et sur la *sécurité des données*.
En parallèle, je voudrais également améliorer mes compétences dans la création d'interfaces plus ergonomiques et réactives.

Si l'opportunité se présente, j'aimerais compléter mes études par une formation en automatisation des processus et en cybersécurité, afin de mieux comprendre le fonctionnement et les usages de l'IA dans ce cadre. Mon objectif est de poursuivre mes études jusqu'à un niveau Bac+5.

Conclusion

Synthèse des principaux points

Au cours de ce stage, j'ai participé à la conception et au développement d'une application permettant l'automatisation et la centralisation de données issues de ***LinkedIn*** via ***n8n***, ***Unipile*** et un frontend développé avec ***Streamlit***.

J'ai travaillé sur toutes les étapes du cycle de vie d'une application :

- ***Backend :***
  Mise en place et configuration de workflows n8n, automatisation de la récupération et de l'enregistrement des messages, gestion de webhooks et intégration d'API.
- ***Base de données :***
  Assistance à la modélisation, à la création et à la gestion de tables relationnelles sur Supabase, conception d'une *fonction SQL stockée* *(cf Annexe 7 : La fonction SQL stockée dans Supabase)* pour faciliter l'accès aux données.
- ***Frontend :***
  Développement d'une interface intuitive avec Streamlit pour afficher les conversations, envoyer des messages LinkedIn et gérer l'authentification via Unipile.
- ***Tests et validations :***
  Mise en place de scénarios d'utilisattion complets afin de garantir la fiabilité et l'ergonomie du prototype.

Ce stage m'a permis de renforcer mes compétences techniques (API, automatisation, gestion de bases de données, développement web) et d'acquérir une meilleure autonomie dans la gestion de projet.

Réponse à la problématique

La problématique de départ était de pouvoir exploiter efficacement les données LinkedIn collectées par n8n, qui ne dispose pas d'interface utilisateur intégrée.
La solution mise en place a consisté à développer une application complète reposant sur trois volets :

- ***Automatisation :*** Mise en place d'un système de récupération et d'enregistrement des conversations et messages dans Supabase.
- ***Interface utilisateur :*** Création d'un frontend ergonomique avec Streamlit permettant de consulter et envoyer des messages.
- ***Intégration des services :*** Assurer une communication fluide entre Unipile, n8n, Supabase et Streamlit.

Le projet fournit un prototype fonctionnel qui répond au besoin d'automatisation, de centralisation et d'accessibilité des données, tout en ouvrant la voie à des évolutions futures (intégration multi-plateformes, analyse big data par intelligence artificielle, optimisation de l'interface utilisateur).

Remerciements

Je tiens à remercier chaleureusement M. Victor Langlois, dirigeant de l'entreprise OPS IMPERIUM, pour m'avoir accueilli et encadré tout au long de ce stage.
Son accompagnement et ses conseils m'ont permis de progresser tant sur le plan technique que méthodologique, et de découvrir un domaine fascinant, en pleine évolution.
Je le remercie également pour la confiance qu'il m'a accordée et pour m'avoir offert l'opportunité de travailler sur un projet concret et complet, particulièrement enrichissant et porteur de nombreux défis tout au long de sa réalisation.

Annexes

Annexe 1 : Déploiement via Docker

```
#utilisation de l'image officielle Nginx
FROM nginx:alpine
#supprimer la config par défaut de Nginx
RUN rm -rf /usr/share/nginx/html/*
#copier mon app dans le dossier web de Nginx
# il ne faut pas utiliser le COPY ici si j'utilise "volumes" dans
docker-compose.yml
# ou COPY ou volumes
COPY html/ /usr/share/nginx/html/
COPY js/ /usr/share/nginx/html/js/
#exposer le port 80
EXPOSE 80
#lancement de Nginx en mode premier plan => sinon Nginx se lance en
arrière plan
CMD ["nginx", "-g", "daemon off;"]
```

Annexe 2 : Nœuds utilisés dans le *Mini-projet English*

- **Schedule Trigger** : définir le jour, l'heure d'exécution du workflow.
- **Formulation la demande (Edit Fields)** : spécifier le contenu souhaité.
- **AI Agent** : demander à l'IA de générer le texte.
- **Story (Edit Fields)** : récupérer l'histoire produite par l'IA.
- **Transformation du texte en audio (HTTP Request)** : transformer le texte en audio via l'API *Text-to-Speech* de Google Cloud.
- **Merge** : combiner le texte et l'audio.
- **Texte et mp3 (Function) :** convertir le contenu audio encodé en Base64 en un fichier MP3 et le renvoyer avec le texte.
- **Send a message** : envoyer le texte et le fichier audio par e-mail à l'adresse définie.

Annexe 3 : Nœuds utilisés dans le workflow *Linkedin_chats*

- ***Edit fields (set chat)*** : gestion de pagination de l'API (paramètres *« limit »* et *« offset »)*
- ***HTTP request :*** configuration de la requête HTTP vers l'API Unipile (endpoint chats) afin de pouvoir récupérer les conversations.
  - Création d'un compte Unipile et connexion à LinkedIn (préciser le chemin de la requête : *« /chats »*)
  - Puis mise en place d'une authentification dans l'application et création d'un webhook ***(détails précisés dans la section « Authentification via Unipile » page 17).***
- ***Split Out Chat :*** traitement de chaque chat individuellement plutôt qu'en tableau complet.
- ***Code :*** transtypage du *champ folder* (initialement de type string) en tableau.
- ***Transform Chat :*** préparation des données récupérées depuis l'API Unipile avant insertion dans Supabase.
- ***Loop Over Items :*** traitement séquentiel des chats.
  - ***Execute SQL query :*** vérification de la présence du chat dans la base (via « *row_id », « id »* et *« account_id »*) :

    ```
    SELECT row_id, id, account_id
    FROM linkedin_chats
    WHERE id = '{{ $json.chat_id }}'
    LIMIT 1;
    ```
  - ***Merge :*** combinaison des sorties de *Transform Chat* et de la *requête SQL*, pour éviter les erreurs en cas de données manquantes.
  - ***Switch :*** Si *« row_id »* existe, le chat est déjà en base; sinon, il est inséré .
  - ***HTTP request (Save chats to Supabase):*** insertion des chats dans la table correspondante en configurant avec la bonne authentification.
  - ***Execute SQL query :*** récupération de *« row_id »* généré par Supabase.
- ***Merge :*** afin de pouvoir transmettre les données nécessaires comme le *« row_id »* au subworkflow suivant.
- ***If :*** filtrage pour éviter l'envoi de tables vides.
- ***Execute workflow :*** appel du subworkflow de traitement des messages.

Annexe 4 : Nœuds utilisés dans le workflow *Linkedin_messages*

- ***When Executed by Another Workflow :*** déclenchement par le *workflow parent.*
- ***HTTP request :*** configuration de la requête HTTP vers l'API Unipile (endpoint message) afin de pouvoir récupérer les messages d'une conversation, en utilisant l'authentification de l'API Unipile et le chemin personnalisé : « */chats/{{ $json.chat_id }}/message ».*
- ***Split Out messages :*** traitement de chaque message individuellement.
- ***Transform Message:*** préparation des données récupérées avant insertion dans Supabase.
- ***Execute SQL query :*** vérification de la présence du chat dans la base via « *row_id », « id »* et *« account_id »*:

  SELECT row_id, id, account_id
  FROM linkedin_chats
  WHERE id = '{{ $json.chat_id }}'
  LIMIT 1;
- ***Merge :*** combinaison des sorties de *Transform Message* et de la *requête SQL*, pour éviter les erreurs en cas de données manquantes.
- ***If :*** filtrage pour éviter les doublons dans la table de *linkedin_messages*.
- ***Supabase :*** insertion des messages dans la base de données.
- ***Code (récup senderId et accounId) :*** extraction et transmission de données nécessaires au subworkflow suivante.
- ***Execute workflow :*** appel du subworkflow de traitement des personnes liées aux chats.

Annexe 5 : Nœuds utilisés dans le workflow *Linkedin_people*

- ***When Executed by Another Workflow :*** déclenchement par le *workflow linkedIn_messages.*
- ***Code (filtrage par providerId)***
- ***HTTP request :*** configuration de la requête HTTP vers l'API Unipile (endpoint « */users/{provider_id}* ») afin de pouvoir récupérer les données des profils avec l'authentification.
- ***Transform People:*** préparation des données récupérées depuis l'API Unipile avant insertion dans Supabase.
- ***Execute SQL query :*** vérification de la présence de l'utilisateur dans la base (via « *row_id », « id »* et *« account_id »*) :

  SELECT row_id, id, account_id
  FROM linkedin_people
  WHERE id = '{{ $json.id }}'
  LIMIT 1;
- ***Merge :*** combinaison des sorties de *Transform People* et de la *requête SQL*, pour éviter les erreurs en cas de données manquantes.
- ***If :*** filtrage pour éviter les doublons dans la table concernée.
- ***Supabase :*** insertion des utilisateurs dans la base.

Annexe 6 : Document comparatif des technologies et frameworks

1. HTML (HyperText Markup Language)

Avantages	Inconvénients
Facile à apprendre, syntaxe intuitive	Ne permet pas de créer des pages dynamiques
Compatible avec tous les navigateurs	Nécessite CSS et JavaScript pour un rendu complet
Structure sémantique claire	Peu sécurisé seul
Indispensable pour tout site web	Beaucoup de code pour des pages simples

2. CSS (Cascading Style Sheets)

Avantages	Inconvénients
Permet de styliser rapidement une page	Comportement parfois différent selon navigateurs
Séparation du contenu et du design	Pas de logique (boucles, conditions…)
Compatible avec le design responsive	Peut devenir complexe à maintenir
Léger et rapide à charger	Pas sécurisé (code visible et modifiable)

3. JavaScript

Avantages	Inconvénients
Rend les pages interactives et dynamiques	Interprétation différente selon les navigateurs
Large écosystème (React, Vue, etc.)	Vulnérable aux attaques côté client
Facile à intégrer avec HTML/CSS	Peut devenir difficile à maintenir sur gros projets
Exécution rapide côté client	Débogage parfois complexe

4. TypeScript (superset de JavaScript)

Avantages	Inconvénients
Typage statique = moins d'erreurs	Courbe d'apprentissage plus raide
Idéal pour les gros projets	Nécessite compilation vers JavaScript
Meilleure lisibilité et maintenance	Moins flexible que JavaScript pur
Supporté par tous les frameworks JavaScript	Configuration initiale plus lourde

5. JSX (JavaScript XML – utilisé avec React)

Avantages	Inconvénients
Écriture intuitive de composants React	Spécifique à React, peu réutilisable ailleurs
Combine HTML et JavaScript dans un seul fichier	Pas compris nativement par les navigateurs
Facilite la création d'interfaces	Nécessite une transformation de code JSX en Javascript classique (via Babel, etc.)

Framework

React (JavaScript)

- **Avantages :**
  - Écosystème énorme, avec une communauté très active
  - Réutilisation des composants pour un code modulaire
  - Intégration fluide avec de nombreux outils (Next.js, Redux...)
  - Maintenu par Meta, donc bonne pérennité
- **Inconvénients :**
  - Le JSX peut sembler déroutant au début
  - Nécessite souvent d'autres librairies pour une application complète
  - Peut devenir complexe avec la gestion d'état avancée

Vue.js (JavaScript)

- **Avantages :**
  - Très intuitif, idéal pour les débutants
  - Documentation claire et bien structurée
  - Structure basée sur HTML, CSS et JavaScript natifs
  - Intégration facile dans des projets existants
- **Inconvénients :**
  - Moins utilisé dans les grandes entreprises que React
  - L'écosystème Vue 2 vs Vue 3 peut créer de la confusion
  - Moins de plugins que React pour certaines fonctionnalités

Svelte (JavaScript)

- **Avantages :**
  - Pas besoin de Virtual DOM → performance ++
  - Syntaxe ultra simple et élégante
  - Compile en JavaScript natif, donc très rapide
  - Poids léger des bundles (ensembles du code JavaScript final livrés au navigateur)
- **Inconvénients :**
  - Communauté plus petite → moins de ressources
  - Moins de cas d'usage en entreprise, donc moins d'offres professionnelles
  - Nouveaux paradigmes à assimiler

Streamlit (Python)

- **Avantages :**
  - Penser pour les data science et les dashboards (tableau de bord)
  - Syntaxe Python simple → pas besoin d'apprendre JavaScript
  - Déploiement ultra rapide d'applications web
  - Idéal pour le prototypage et les présentations
- **Inconvénients :**
  - Pas conçu pour des applications complexes ou multi-utilisateurs
  - Limité en personnalisation UI comparé aux frameworks JavaScript
  - Moins adapté pour des interfaces très interactives

Angular (framework JavaScript/TypeScript)

- **Avantages :**
  - Framework complet avec architecture solide (MVC, injection de dépendances, routage...)
  - Utilise TypeScript → typage statique, meilleure maintenabilité
  - Idéal pour les applications complexes et à grande échelle (SPA, PWA)
  - Soutenu par Google → mises à jour régulières et documentation riche
  - Outils puissants comme Angular CLI et Angular Material
- **Inconvénients :**
  - Courbe d'apprentissage plus raide que celle de React ou Vue.js
  - Moins adapté aux petites applications ou projets simples
  - Mises à jour fréquentes qui peuvent casser la compatibilité
  - Performances parfois moins optimales pour les applications légères

Pandas (Python)

- **Avantages :**
  - Manipulation de données ultra puissante (DataFrames, Series)
  - Syntaxe intuitive et proche du langage naturel
  - Large écosystème et compatibilité avec NumPy, Matplotlib, Scikit-learn…
  - Idéal pour le nettoyage, l'exploration et la transformation de données
  - Documentation riche et communauté active
  - Compatible avec les modèles IA via intégration avec TensorFlow, PyTorch, etc.
- **Inconvénients :**
  - Moins performant avec très grands volumes de données
  - Peut consommer beaucoup de mémoire
  - Pas conçu pour le calcul distribué (contrairement à Spark)
  - Courbe d'apprentissage pour les fonctions avancées
  - Certaines opérations peuvent être lentes sans optimisation
  - Pas adapté pour le traitement en temps réel ou les pipelines complexes

Annexe 7 : La fonction SQL stockée dans Supabase

```
CREATE OR REPLACE FUNCTION get_messages()
RETURNS TABLE (
  message_id text,
  content text,
  sender text,
  full_name text,
  linkedin_url text,
  profile_picture_url text,
  created_at timestamp,
  chat_row_id text,
  is_from_me boolean,
  attendee_provider_id text,
  attendee_name text,
  account_id text,
  chat_id text,
  provider_id text
)
LANGUAGE SQL AS $$
  SELECT
    lm.id AS message_id,
    lm.content,
    lm.sender,
    coalesce(lp.full_name, lp2.full_name) AS full_name,
    coalesce(lp.linkedin_url, lp2.linkedin_url) AS linkedin_url,
    coalesce(lp.profile_picture_url, lp2.profile_picture_url) AS
profile_picture_url,
    lm.created_at,
    lm.chat_row_id::text AS chat_row_id,
    lm.is_from_me,
    lc.attendee_provider_id,
    lp2.full_name AS attendee_name,
    lm.account_id,
    lm.chat_id,
    lm.provider_id
  FROM linkedin_messages lm
//récuperer les infos de l'expéditeur du message = >relier chaque
message à son expéditeur :
  LEFT JOIN linkedin_people lp ON lm.sender = lp.id
  LEFT JOIN linkedin_chats lc ON lm.chat_row_id = lc.row_id
// récupérer les infos du participant à la conversation => relier
chaque conversation à l'autre participant  (autre que expéditeur)
  LEFT JOIN linkedin_people lp2 ON lc.attendee_provider_id = lp2.id
  ORDER BY lm.created_at DESC;
$$;
```

Figure 1: Diagramme de Gantt

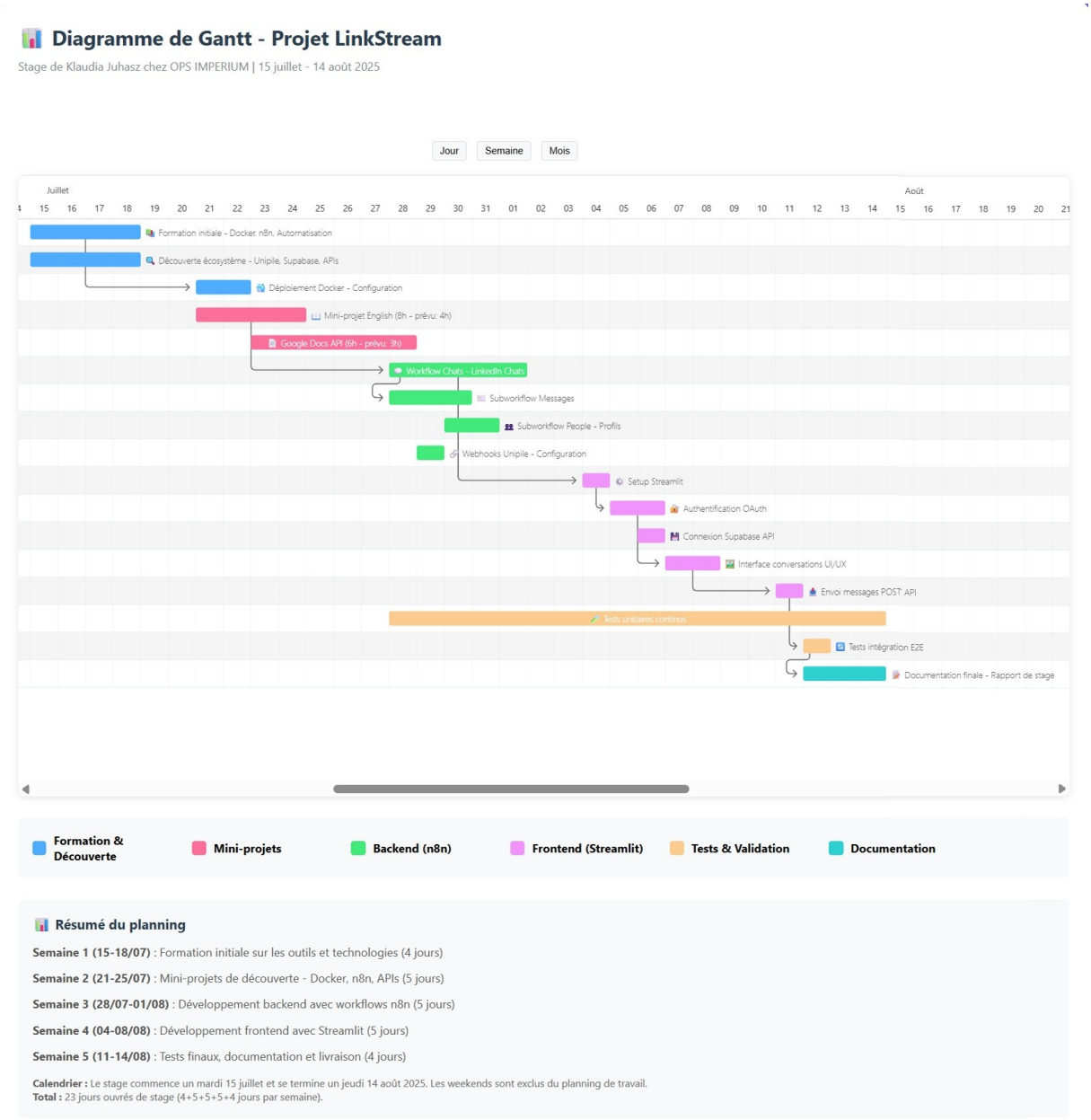


Figure 2: Déploiement via Docker

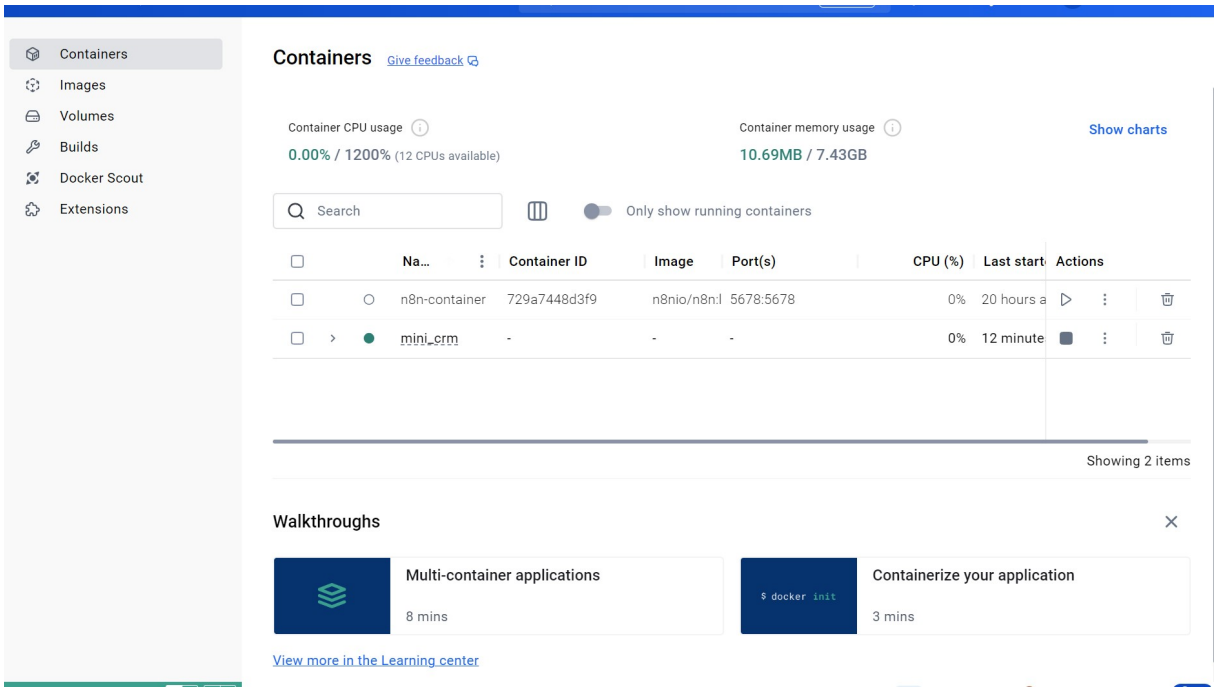


Figure 3: Mini-projet English

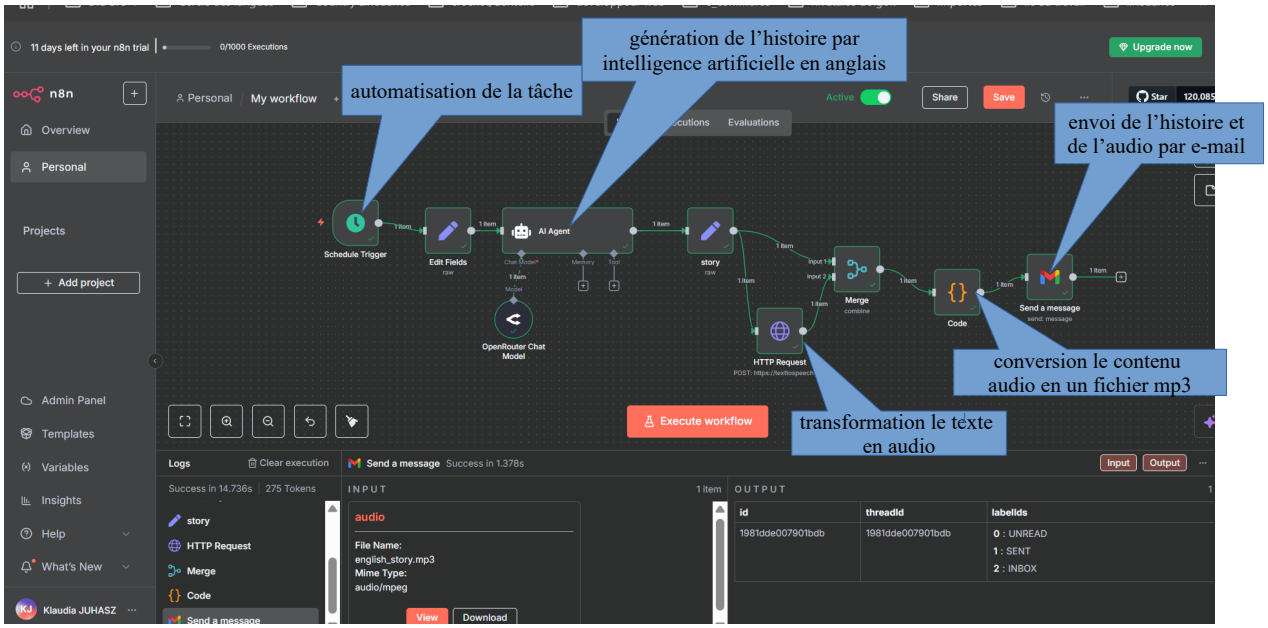


Figure 4: Mini-projet Google Docs API

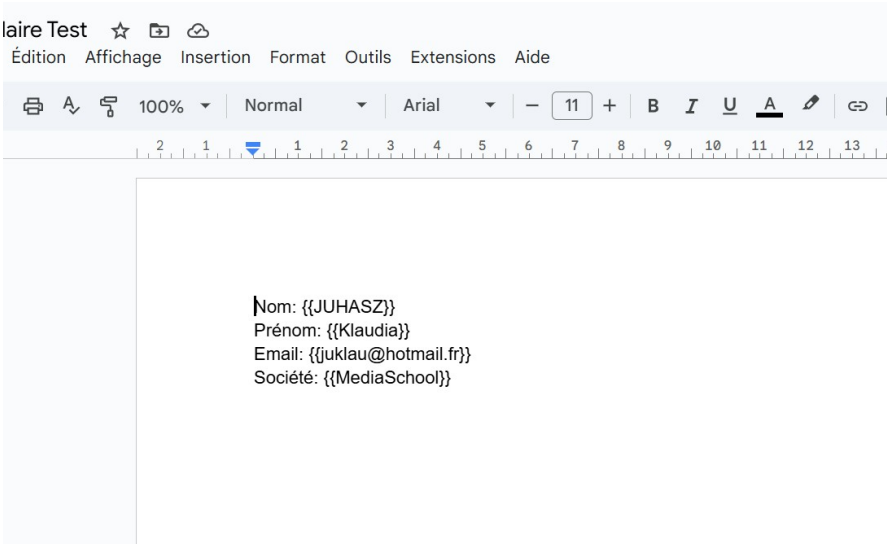


Figure 5: Workflow Linkedin Chat

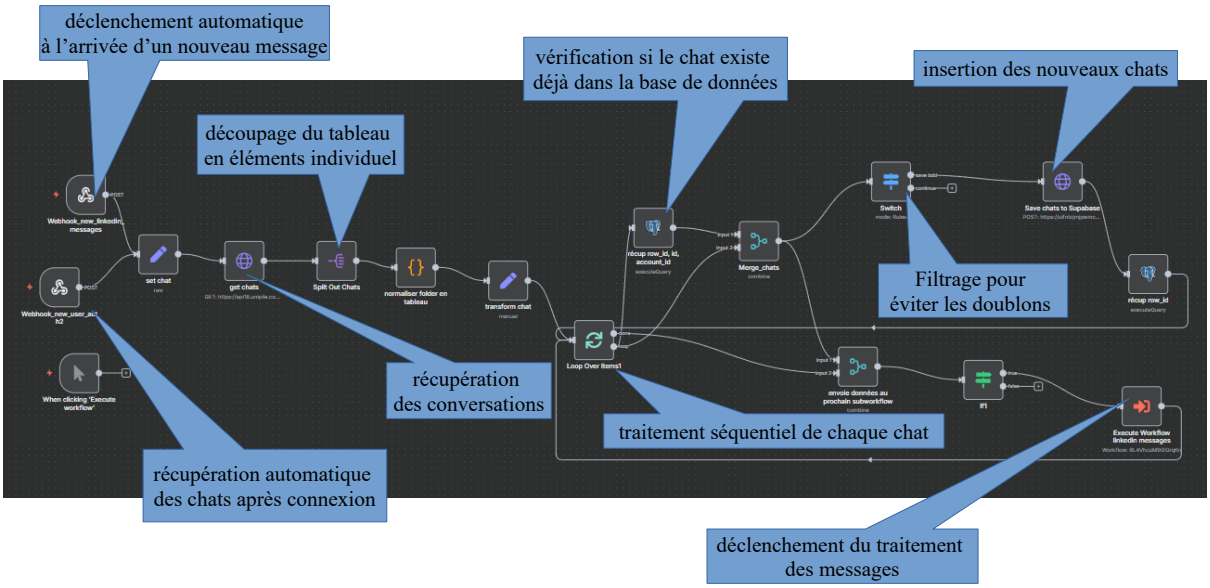


Figure 6: Workflow register webhook Unipile

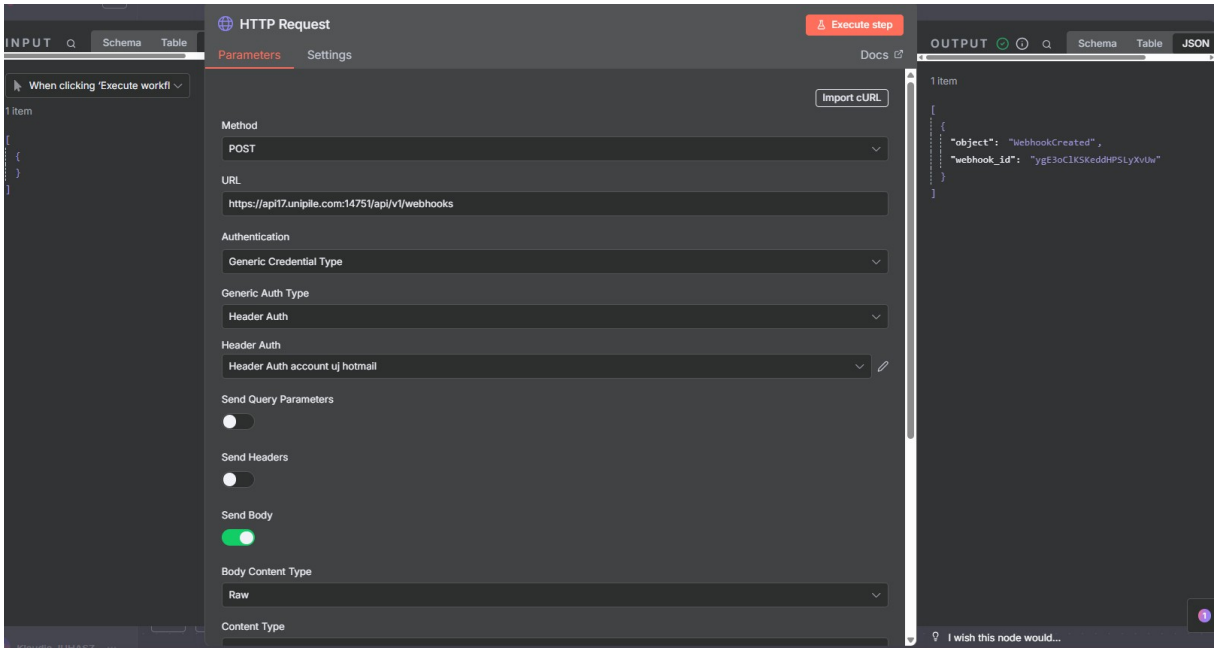


Figure 7: Subworkflow linkedin_messages

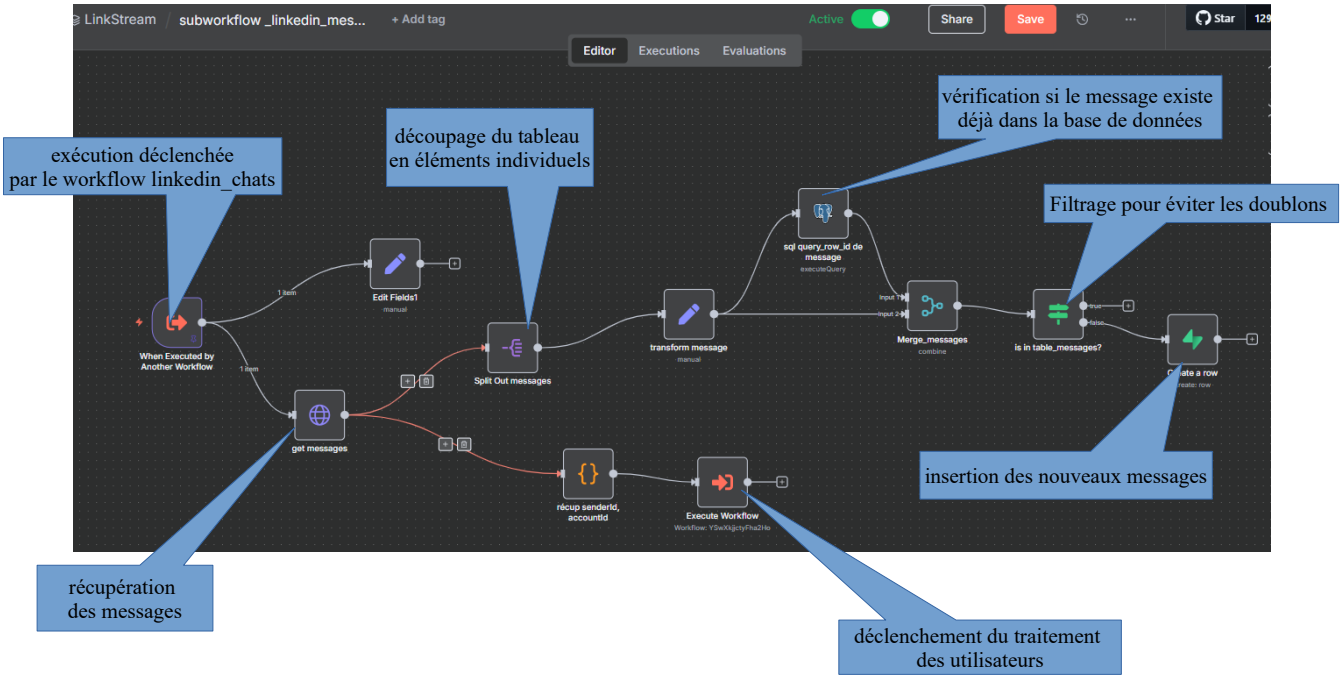


Figure 8: Subworkflow linkedin_people

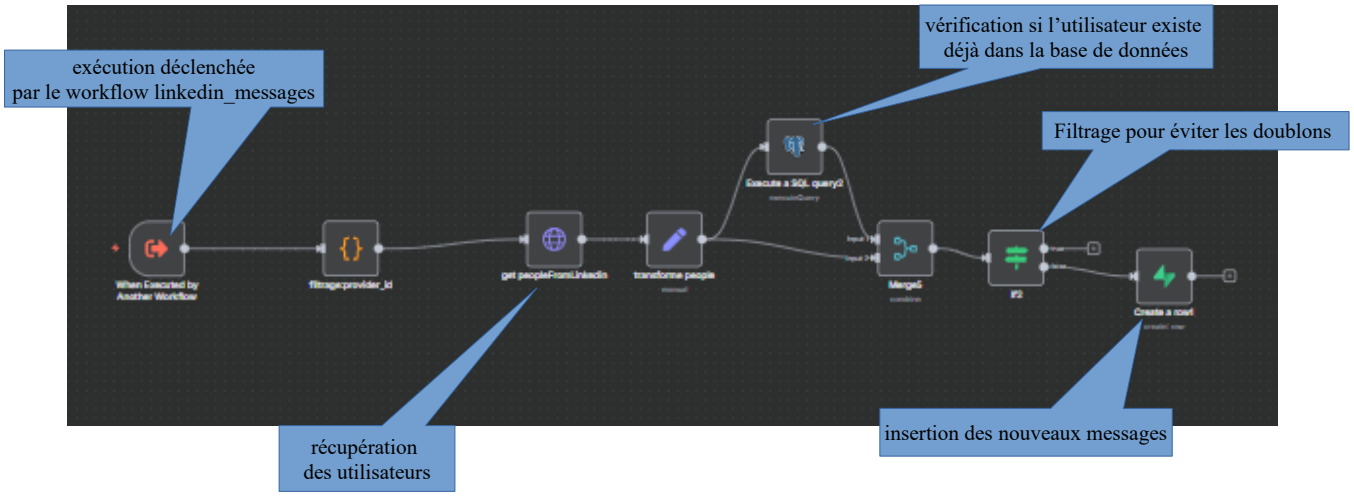


Figure 9: Création webhook par Unipile

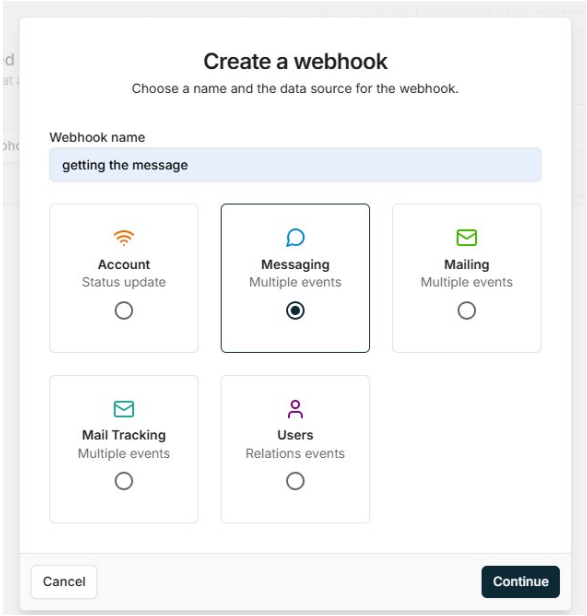


Figure 10: Authentification par Unipile

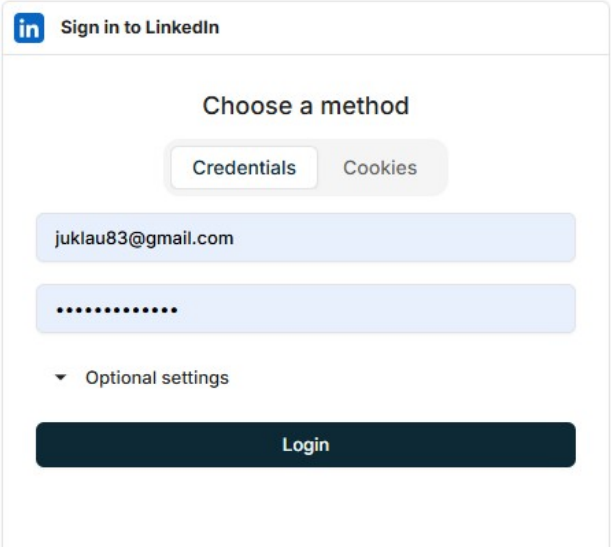


Figure 11: Redirection vers l'application LinkStream

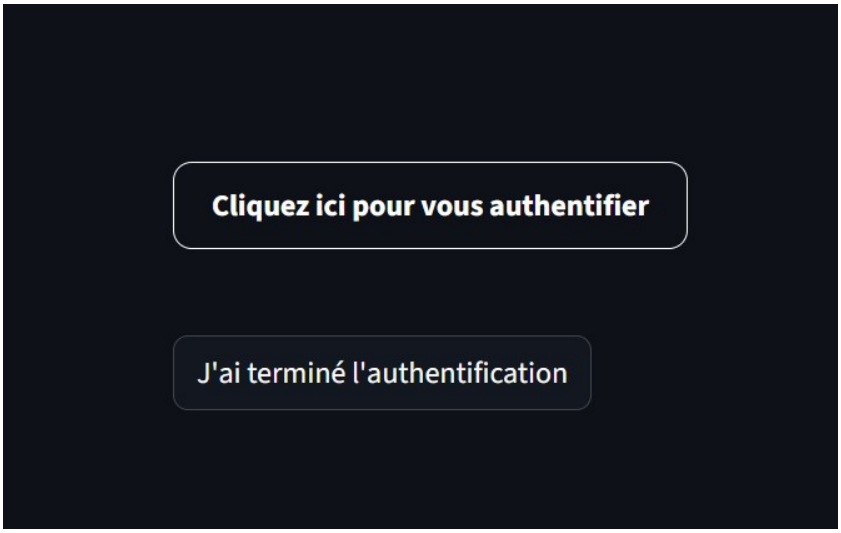


Figure 12: Affichage dynamique

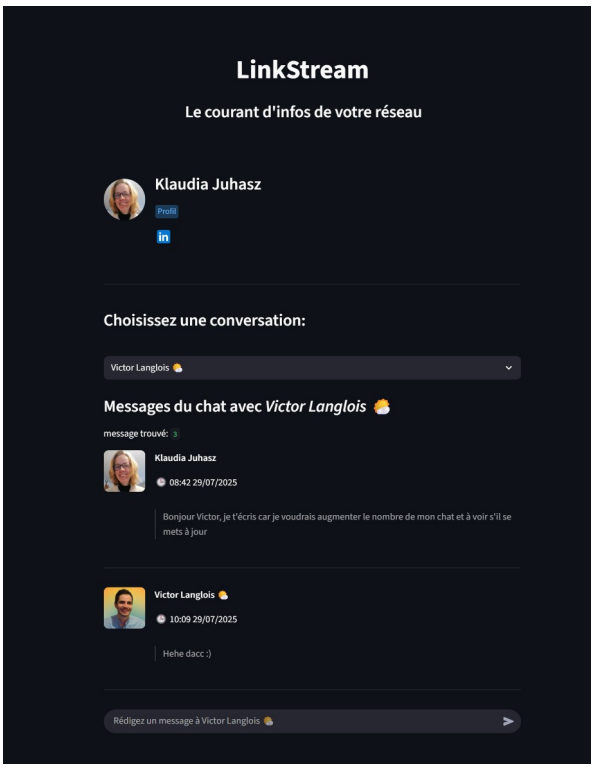


Figure 13: Affichage des conversations

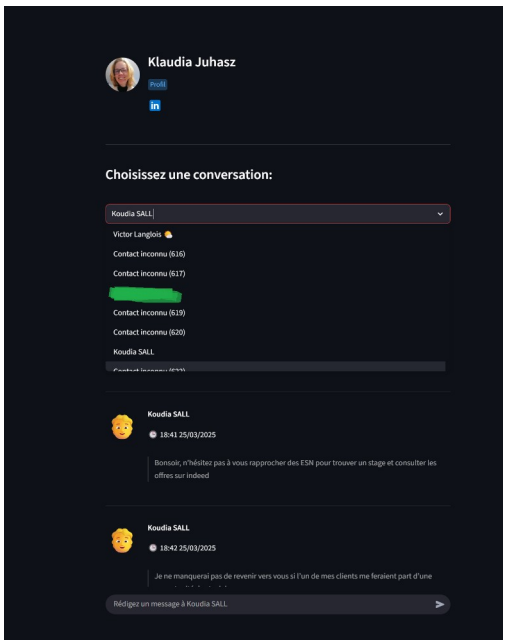


Figure 14: MCD de la base de données Supabase, réalisé avec Looping

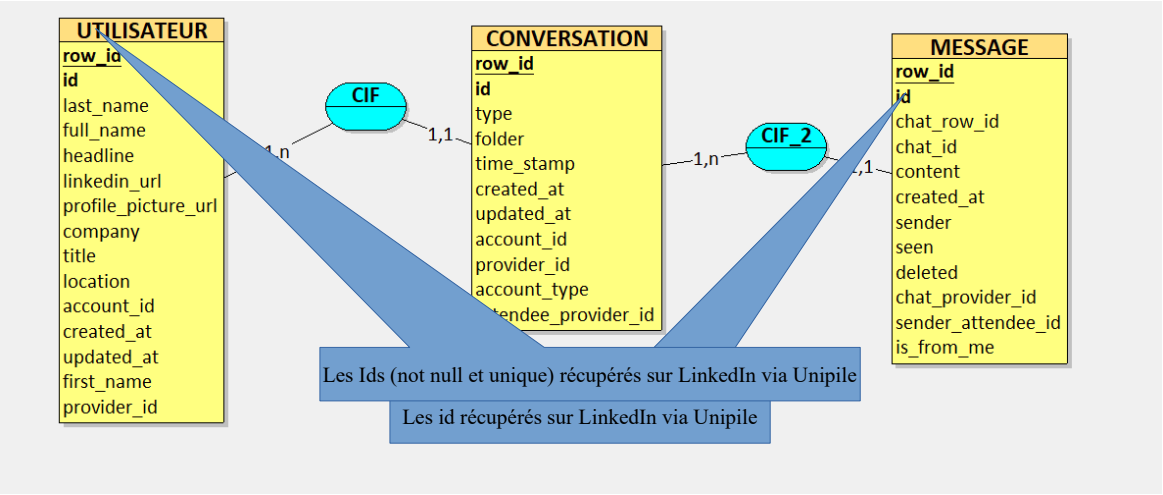
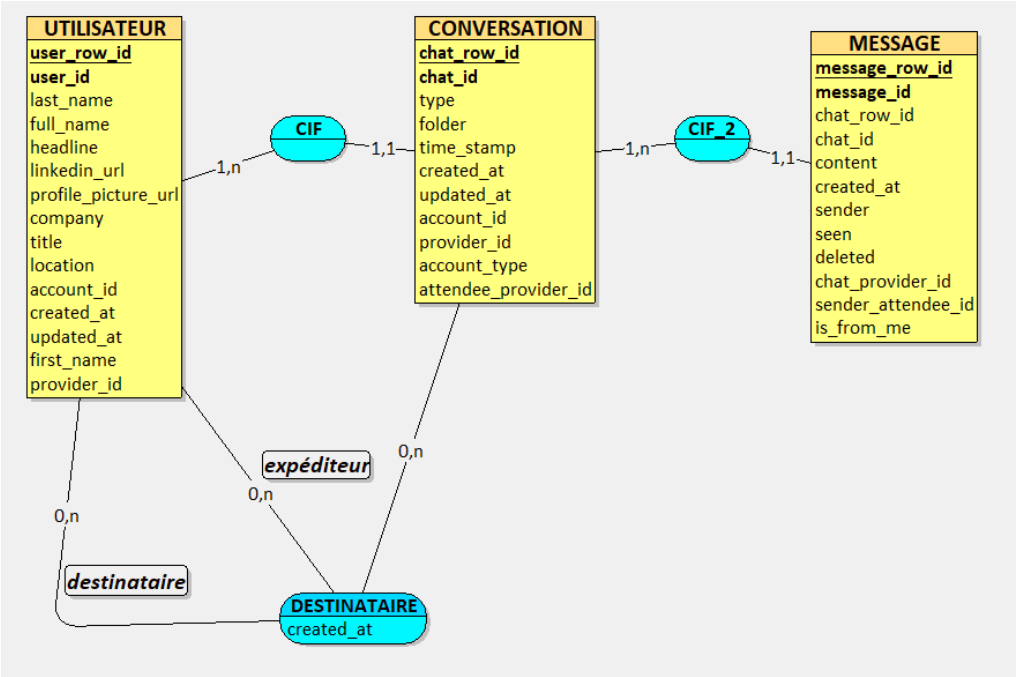


Figure 15: MCD (proposition d'évolution), réalisé avec Looping



Bibliographie

- ***Docker***
  - Tutoriel YouTube – *Introduction au Dockerfile :* https://www.youtube.com/watch?v=P3uvEikM_T0
  - Tutoriel YouTube – *Déployer un environnement Docker* https://www.youtube.com/watch?v=ES4BcZcsBdU
  - Tutoriel YouTube – *Introduction to Automation :* https://www.youtube.com/watch?v=4BVTkqbn_tY
- ***OpenAI***
  - OpenAI (2025). *Utilisation de modèles d'intelligence artificielle pour la génération de texte, l'explication de codes et la clarification de définitions*. Disponible sur : https://openai.com
- ***Google Cloud***
  - Google (2025). *Text-to-Speech – Documentation officielle*. Disponible sur : https://cloud.google.com/text-to-speech?hl=fr&_gl=1*177fsde*_ga*MTY3ODUxMTk2NC4xNzUyNTY3NjMz*_ga_WH2QY8WWF5*czE3NTI4MjcwNzckbzEwJGcxJHQxNzUyODI3NDMyJGoyMiRsMCRoMA..
  - Google Developers (2025). *Google Docs API Reference*. Disponible sur : https://developers.google.com/workspace/docs/api/reference/rest?hl=fr
- ***n8n***
  - n8n. *Documentation officielle – Installation Docker.* Disponible sur : https://docs.n8n.io/hosting/installation/docker/#prerequisites
  - n8n. *Documentation – Flow Logic et Looping*. Disponible sur : https://docs.n8n.io/flow-logic/looping/#node-exceptions
  - n8n. *Cours vidéo officiels*. Disponible sur : https://docs.n8n.io/video-courses/
  - Tutoriel YouTube – *Découverte de n8n* : https://www.youtube.com/watch?v=sh6K862NAkk
  - Tutoriel YouTube – *Automatisations avec n8n : https://www.youtube.com/watch?v=HLmni373qc4*
- ***Supabase***
  - Supabase. *Documentation officielle*. Disponible sur : https://supabase.com/docs

- ***Python***
  - Python Software Foundation. *Documentation officielle Python 3 – Datetime*. Disponible sur : https://docs.python.org/3/library/datetime.html#datetime.datetime.fromtimestamp

- ***Unipile***
  - Unipile. *Site officiel*. Disponible sur : https://www.unipile.com/fr/
  - Unipile. *Getting Started – Documentation développeurs*. Disponible sur : https://developer.unipile.com/docs/getting-started
  - Unipile. *Connect Accounts – API Documentation*. Disponible sur : https://developer.unipile.com/docs/connect-accounts

- ***Streamlit***
  - Streamlit. *Documentation officielle*. Disponible sur : https://docs.streamlit.io/
  - Streamlit. *Cheat Sheet – Quick Reference*. Disponible sur : https://docs.streamlit.io/develop/quick-reference/cheat-sheet
  - Tutoriel YouTube – *Découverte de Streamlit : https://www.youtube.com/watch?v=ITF1IzvfHmA*

- ***Pandas***
  - Pandas. *Site officiel*. Disponible sur : https://pandas.pydata.org/
  - Pandas. *API Reference – DataFrame.dropna*. Disponible sur : https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html